IT UNIVERSITY OF CPH

Report group P

0.0.1 Anton Yakovenko anya@itu.dk

0.0.2 Janusz Bekas jdbe@itu.dk

0.0.3 Mengdi Liao menl@itu.dk

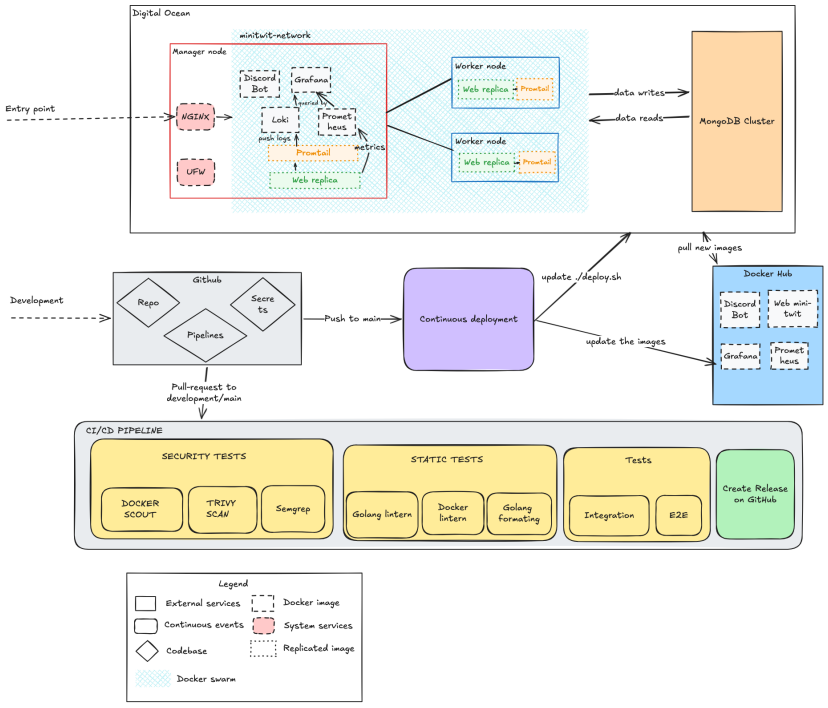
0.0.4 Viktor Horvath vhor@itu.dk

0.1 Introduction

Itu-minitwit is an evolved version of the MinitTwit project originally written in python and Flask. It was re-written in GoLang with the Gorilla webtoolkit framework. We use MongoDB for our database. Loki, Prometheus, Promtail and Grafana are used as a monitoring stack.

The project's codebase is located on GitHub and utilizes GitHub Actions for CI/CD pipeline. We are using DigitalOcean as our host for Virtual machines and a managed database.

0.2 System perspective



System architecture view

The production runtime is Docker Swarm on DigitalOcean. The Swarm has one manager node, minitwit, and two worker nodes, minitwit-web-1 and minitwit-web-2. The remote stack consists of webserver, prometheus, grafana, loki, promtail, and discordbot services on the overlay network minitwit-network. Manager node consists of a monitoring stack (prometheus, grafana, loki, a promtail replica and webserver replica). Worker nodes have webserver replicas and promtail replicas.

The web application and Discord bot both connect to the database through MONGO_URI.

Monitoring is implemented using Grafana for visual display of graphs and logs.

Logs are being created in webserver containers, then Promtail runs as a global Swarm service and ships Docker container logs to Loki.

Graphs are created by prometheus querying the webserver system and passing the information to grafana that interprets information and displays them as informational graphs.

Necessary environment variables are stored in GitHub Secrets.

0.3 Dependencies

Higher-Level Dependencies
DigitalOcean Droplets: Cloud VMs that host the production Docker Swarm nodes.
DigitalOcean Managed MongoDB: Hosted production MongoDB database used by the app and bot.
Docker: Runs the application and infrastructure services in containers.
Docker Compose: Defines and runs the local development environment.
Docker Swarm: Orchestrates the remote production stack across multiple droplets.
Docker Hub: Registry where production images are pushed and pulled from.
Vagrant: Current tool for provisioning DigitalOcean droplets.
vagrant-digitalocean: Vagrant provider plugin/box for creating DigitalOcean droplets.
GitHub Actions: CI/CD platform for tests, static analysis, image builds, releases, and deployments.
Nginx: Reverse proxy in front of the app and Grafana.
Certbot: Automates Let's Encrypt certificate setup and renewal.
Let's Encrypt: Issues TLS certificates.
UFW: Host firewall configured during provisioning.
Prometheus: Collects application and service metrics.
Grafana: Displays dashboards for metrics and logs.
Loki: Stores and queries logs.
Promtail: Collects container logs and ships them to Loki.
rsyslog: Older/auxiliary centralized log collection component.
Pandoc / LaTeX: Builds the project report PDF.
Firefox / geckodriver / Selenium: Runs browser-based UI tests.
Docker Scout: Scans Docker images for vulnerabilities.
Trivy: Performs container vulnerability scanning.
Semgrep: Static analysis for security/code patterns.
CodeQL SARIF upload: Uploads scan results to GitHub code scanning.
golangci-lint: Runs Go linting.
Hadolint: Lints Dockerfiles.

Container / Image Dependencies
'mongo:8.0': Local MongoDB database image.
'grafana/grafana:12.1': Base image for the custom Grafana image.
'prom/prometheus:v3.5.1': Base image for the custom Prometheus image.
'grafana/loki:latest': Loki log storage/query service image.
'grafana/promtail:latest': Promtail log shipper image.
'ubuntu:24.04': Base image for the rsyslog container.
'golang:1.25.10': Build image for compiling the Go web app.
'alpine:3.23': Lightweight runtime image for the Go web app.
'pandoc/latex:3.6': CI image used to build the report PDF.
'curlimages/curl': Small image used for HTTP checks in older/local helper

flows.

Main Go App Dependencies
‘github.com/gorilla/mux‘: HTTP router for app routes.
‘github.com/gorilla/sessions‘: Cookie/session management.
‘github.com/prometheus/client_golang‘: Exposes Prometheus metrics from the app.
‘go.mongodb.org/mongo-driver‘: MongoDB client driver.
‘golang.org/x/crypto‘: Crypto utilities, including password hashing support.

Discord Bot Dependencies
‘github.com/bwmarrin/discordgo‘: Discord API client library.
‘go.mongodb.org/mongo-driver‘: MongoDB client driver used by the bot.

Important Go Indirect Dependencies
‘github.com/gorilla/securecookie‘: Secure cookie encoding used by Gorilla sessions.
‘github.com/gorilla/websocket‘: WebSocket support, used indirectly by Discord integration.
‘github.com/prometheus/client_model‘: Prometheus metric data model.
‘github.com/prometheus/common‘: Shared Prometheus helpers.
‘github.com/prometheus/procfs‘: Reads Linux process metrics for Prometheus.
‘github.com/klauspost/compress‘: Compression support.
‘github.com/golang/snappy‘: Snappy compression support, used by MongoDB-related code.
‘github.com/xdg-go/scram‘: SCRAM authentication support for MongoDB.
‘github.com/xdg-go/pbkdf2‘: Password-based key derivation used in auth flows.
‘github.com/xdg-go/stringprep‘: String preparation used in authentication protocols.
‘google.golang.org/protobuf‘: Protocol Buffers support.
‘golang.org/x/sys‘: Low-level OS/system calls.
‘golang.org/x/text‘: Text encoding and normalization utilities.
‘golang.org/x/sync‘: Extra concurrency helpers.
‘golang.org/x/net‘: Extended networking libraries.

Python / Test Dependencies
‘pytest‘: Python test runner.
‘pymongo‘: Python MongoDB client used by tests.
‘selenium‘: Browser automation for UI tests.
‘requests‘: HTTP client used by simulator/test tooling.

GitHub Actions Dependencies
‘actions/checkout‘: Checks out repository code in CI.
‘docker/login-action‘: Logs CI into Docker Hub.
‘docker/setup-buildx-action‘: Enables Docker Buildx in CI.

‘docker/build-push-action‘: Builds and pushes Docker images.
‘actions/setup-go‘: Installs/configures Go in CI.
‘actions/setup-python‘: Installs/configures Python in CI.
‘actions/upload-artifact‘: Uploads generated artifacts like the report PDF.
‘browser-actions/setup-firefox‘: Installs Firefox for UI tests.
‘docker/scout-action‘: Runs Docker Scout vulnerability checks.
‘aquasecurity/trivy-action‘: Runs Trivy scans.
‘golangci/golangci-lint-action‘: Runs Go linting in CI.
‘returntocorp/semgrep-action‘: Runs Semgrep static analysis.
‘softprops/action-gh-release‘: Creates GitHub releases.
‘zwaldowski/semver-release-action‘: Handles semantic version release automation.
‘zwaldowski/match-label-action‘: Checks labels used for release/version decisions.

0.4 Process perspective

To manage the workflow, we have created a Discord channel where we discuss our ideas and current problems and how to solve them. During the evolution process, we handled bugs that arose along the way. We gathered them on github issues, from where we had a clear view for them and assigned the person responsible for the fix. Once the solution was found, developers created a pull request to merge their branch with the fix into the development branch, where tests from CI/CD pipeline were run. After the tests were marked as successful PR required at least one review from the team member to merge into the development branch. Then we tried to create a PR from development to main every week to have continuous weekly releases. We kept naming conventions for new branches which we described in the readme file.

0.5 CI/CD Pipeline: Stages and Tools

Our CI/CD pipeline ensures security and quality at every stage before deployment. The automated workflow consists of four main stages:

1. Security Tests This stage forms the first line of defense, scanning the code and environment for vulnerabilities.

- Semgrep (SAST): Scans the raw source code to detect bad coding patterns, hardcoded secrets, and OWASP Top 10 vulnerabilities.
- Trivy: Scans project dependencies and OS packages for known vulnerabilities (CVEs).
- Docker Scout: Analyzes the compiled Docker images, generating an SBOM and ensuring the final artifact is free of critical container vulnerabilities.

2. Static Tests This stage enforces code quality, readability, and best practices without running the application.

- Golang Linter: Detects syntax errors, dead code, and potential bugs in the Go source code.
- Docker Linter: Validates Dockerfile configurations to ensure images are optimized and follow security best practices.
- Golang Formatting: Ensures consistent code styling across the team.

3. Tests (Dynamic) This stage runs the compiled application to verify system behavior.

- Integration Tests: Runs provided simulator, testing thousands of user having interactions with our application

- End-to-End (E2E) Tests: Simulate real user interactions from start to finish, ensuring the entire ecosystem (UI, backend, and DB) functions as expected.

4. Deployment and Release Triggered only when all previous stages pass successfully.

- Create Release on GitHub: The pipeline automatically generates a new version tag (e.g., v1.0.2), versions are saved in our repo and if we would decide that we want to return to previous deployment, we can easily make it

Monitoring
- We measure time respond of different endpoints using different methodologies (P50 Latency, P95 Latency, P99 Latency)
- Total incoming https request for endpoints
- Overall error rate and 4xx and 5xx responds for different endpoints
- Request / success / error rate by different endpoints

Logging
We are logging failures for different endpoints inside webserver containers. We use promtail to go through the docker container logs and aggregate them for grafana.
Security
We added following following technologies:
- **UFW Firewall (Perimeter Defense):** The host firewall is strictly configured to deny unauthorized external traffic. Public access is restricted entirely to secure management via OpenSSH and encrypted web traffic via the `Nginx Full` profile (which opens HTTP Port 80 and HTTPS Port 443).
- **Nginx & HTTPS:** Nginx acts as a reverse proxy, buffering the public internet from our internal services. All data in transit is fully encrypted using HTTPS (SSL/TLS).
- **Password Hashing:** User credentials are cryptographically hashed before being stored in the database, ensuring no plaintext passwords are ever saved.
- **Automated CI/CD Scans:** Our DevSecOps pipeline actively blocks vulnerabilities before deployment using **Semgrep** (source code analysis), **Trivy** (dependency scanning), and **Docker Scout** (container image CVEs).
- Our docker container images are running as a non-root user except for promtail. We kept promtail user as a root, because it needs more rights to read logs from other containers.

Scaling and availability
We have implemented docker swarm to help with scaling. Furthermore, we are using rolling updates to reduce down time as much as possible. We have 3 replicas for webserver images and a promtail replica for each of webservers to gather all the necessary logs.

0.6 Reflection Perspective

Evolution and refactoring
The app evolved continuously. First we rewrote minitwit in GoLang and MongoDB. Then we introduced the first version of CI/CD pipeline, which only built and published the latest version of the app to the digital ocean host. Then we further improved the quality of the codebase by structuring the project, adding missing features and implementing new ones. Later we improved our CI/CD pipeline to include various security and quality tests.

Operation maintenance:

- After encountering our first errors, we added a custom logging system that collected all errors caught inside the handler and stored them in separate files. Later, we transitioned to specialized technologies, using Promtail to collect log lines from different containers and Loki to store them.

Encountered problems
We have encountered multiple problems during the development and evolution cycle. Main ones are:

- DB non-indexing caused the droplet to be down for multiple days before we even noticed. Droplet ran out of memory and crashed everything. This has caused a downtime of 4-5 days;

- A ransomware bot attack stole some tables due to the DB port being open. MongoDB was reachable from outside the container host on port 27017, and there was no Mongo root username/password configured.

- Virtual machine selection for docker images (some barebones linux distros did not have curl for example) and it failed the ci/cd pipeline;

- During docker swarm creation we forgot to add additional droplets on DigitalOcean and mixed up network and volume names which resulted in a 30 minute downtime;

- In our first logging system we struggled with volumes in our server, where we couldn't export logs and store them outside of the container. Instead of fixing this we moved to promtail and loki.

- Some CI/CD operations couldn't be tested directly in our repository, because changes need to be accepted by other team members. That's why when we were testing github action we used separate repository which was used as sandbox, with full rights

**Use of Generative AI Technologies which we used:

Codex 5.5: for the review of our Docker stack. Powerful models such as Claude 4.7 and Gemini: for code review, troubleshooting, and explaining concepts which we couldn't solve by ourselves. GitHub Copilot: for coding tips during our work process.

Reflections on the work process: Generative AI significantly supported our workflow. With these tools, our project accelerated noticeably, especially when we made errors and the AI models were able to detect them and suggest repair steps. What's more, not only did they help us solve ongoing problems, but they also guided us in understanding new concepts coming from lectures and exercises. However, we had to remain cautious and manually verify the AI's suggestions, as it occasionally lacked the full context of our specific system architecture.